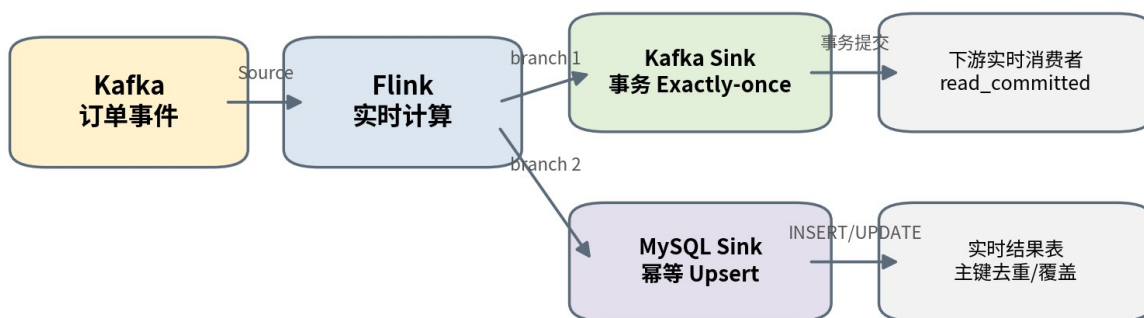


大数据综合项目实战

第 9 讲：把实时结果真正写到外部系统

Kafka Sink + MySQL Upsert + Checkpoint + Delivery Guarantee

第9讲端到端：同一条实时结果，进入不同外部系统时，语义保证并不相同



本讲核心：前两讲已经能“算出结果”。这一讲开始训练真正的交付：结果要离开 Flink，进入 Kafka 或 MySQL，而且必须知道故障重启后会不会丢、会不会重复、怎样去重。

1. 学完这一讲，你必须真正会什么

- 把城市实时销售结果从 `print()` 改成 Kafka Sink 和 MySQL Sink。
- 把订单超时告警从 `print()` 改成 Kafka Topic 和 MySQL 告警表。
- 理解 Sink 不是“最后一行代码”，而是端到端一致性最容易出问题的边界。
- 理解 NONE / AT_LEAST_ONCE / EXACTLY_ONCE 三种交付语义。
- 理解 Flink 内部 Exactly-once State 不等于所有外部系统都自动 Exactly-once。
- 能解释 Kafka 事务 Sink 为什么必须和 Checkpoint 配合。
- 能解释 MySQL 普通 JdbcSink 为什么是 at-least-once，以及为什么主键 + Upsert 能让最终表状态幂等。
- 能设计稳定的业务主键，而不是用随机 UUID 假装“去重”。
- 会用 `read_committed` 验证 Kafka exactly-once 输出，会用 SQL 验证 MySQL 最终表。

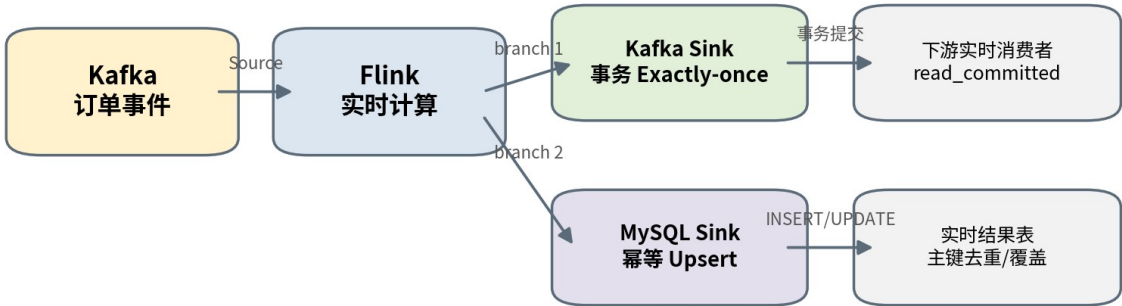
最低闭环：Source 有输入 → Job RUNNING → Checkpoint 持续成功 → Kafka `read_committed` 能看到结果 → MySQL 行数/金额正确 → 解释两种 Sink 的一致性边界。

2. 从 print() 到生产 Sink：为什么这是一个质变

第 7、8 讲使用 result.print() 是为了看懂 Watermark、Window、State、Timer。真实系统不会让运营人员去 TaskManager stdout 找结果，而是必须把结果写到一个“可被其他系统稳定消费”的地方。

教学阶段	输出位置	优点	生产问题
print()	TaskManager stdout	最容易观察	不能查询、不能共享、重启后难追踪
Kafka Sink	Kafka Topic	解耦、可回放、可多消费者	要理解事务、消费隔离、Offset
MySQL Sink	关系型表	业务查询方便、可做主键 Upsert	数据库吞吐、重复写、事务边界

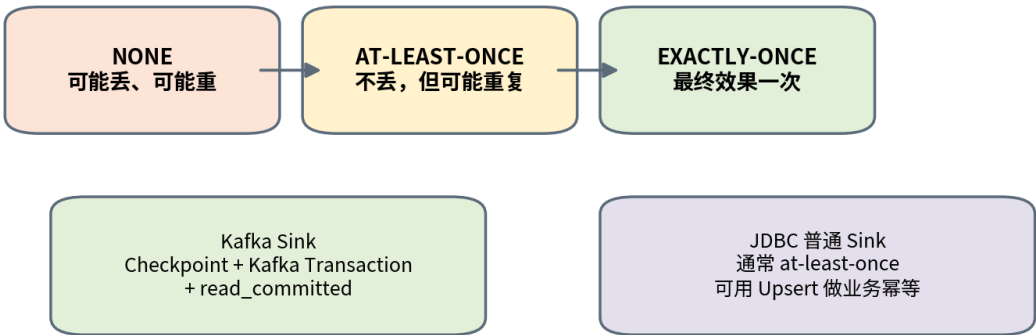
第9讲端到端：同一条实时结果，进入不同外部系统时，语义保证并不相同



核心变化：前面关注“Flink 算得对不对”；现在还必须关注“结果交付出去以后，故障重试会不会造成重复业务影响”。

3. 先把交付语义讲透：Exactly-once 到底是什么意思

交付语义：Flink 内部 Exactly-once ≠ 每一种外部 Sink 都自动 Exactly-once



Exactly-once 更准确地说是“某个业务结果对目标系统的最终可观察效果只发生一次”。它不要求 CPU 指令物理上只执行一次，也不意味着网络永远不重试。

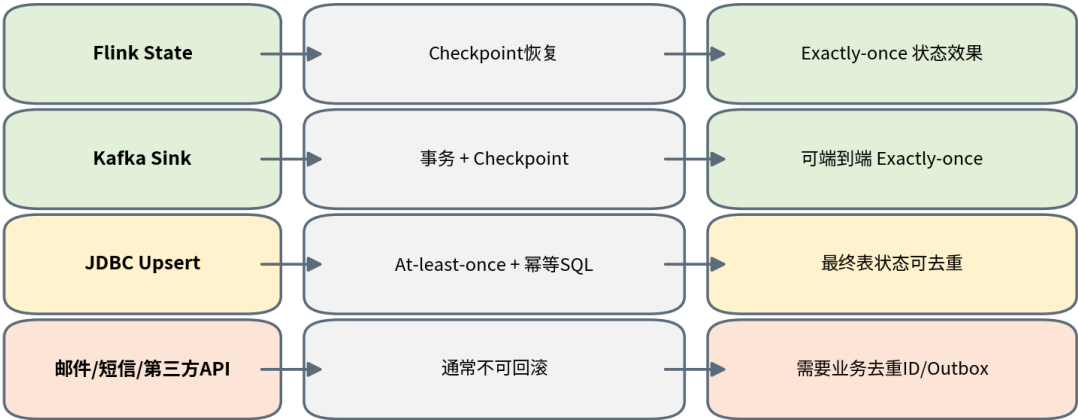
语义	故障时可能发生什么	典型理解
NONE	可能丢，也可能重	只追求低成本，不做强保证
AT_LEAST_ONCE	不丢，但可能重放/重复写	结果必须能接受重复或自己去重
EXACTLY_ONCE	重放可以发生，但最终效果保持一次	依赖 Source、State、Sink 共同参与一致性机制

不要背口号：“Flink 开了 Checkpoint” 只能首先保证 managed state 的一致恢复。外部 Sink 是否 exactly-once，要看 Sink 自己能否和 Checkpoint 协同。

4. 本讲为什么同时写 Kafka 和 MySQL

同一条 Flink 结果进入不同外部系统时，保证能力并不一样。把两种 Sink 放在同一讲，是为了让你真正看到“一致性是端到端属性”。

Exactly-once 是一条端到端链路属性，不是给 Flink 打开一个开关就结束

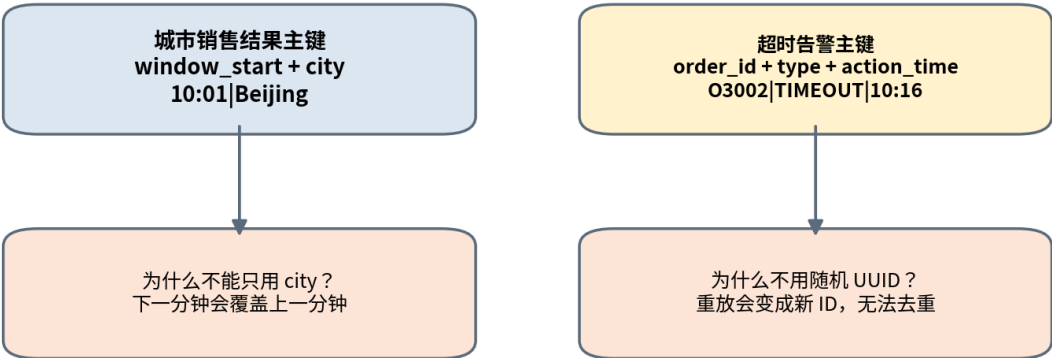


目标	本讲做法	语义
Kafka	KafkaSink + DeliveryGuarantee.EXACTLY_ONCE + transactionalIdPrefix	事务与 Checkpoint 绑定；read_committed 消费可获得 exactly-once 可见结果
MySQL	JdbcSink.sink + 主键 + ON DUPLICATE KEY UPDATE	JdbcSink 本身 at-least-once；重复写通过业务主键 Upsert 幂等化
高级可选	JdbcSink.exactlyOnceSink + XA	可做事务 exactly-once，但需要数据库 XA 配置，复杂度更高，本讲不作为主线

版本线：综合项目继续沿用第 7/8 讲：Flink 1.20.3、Kafka Connector 3.4.0-1.20、JDBC Connector 3.4.0-1.20、Java 11。不要把当前 Flink 2.3 stable 文档里的依赖直接抄进这个旧版本项目。

5. 业务主键：幂等的真正核心

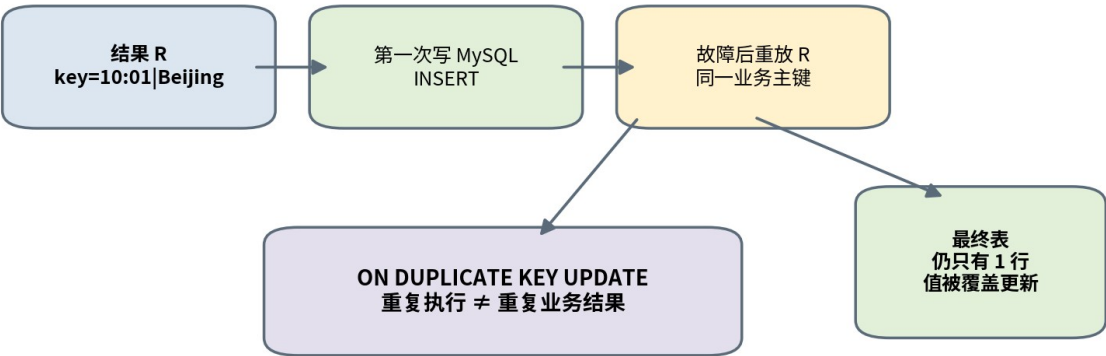
幂等的核心不是“用了数据库主键”，而是“主键必须由业务事实稳定决定”



幂等不是“数据库有 PRIMARY KEY 就行”，而是这个 Key 必须由业务事实稳定决定。故障重放同一个结果时，必须再次生成同一个 Key。

结果	本讲主键	为什么这样设计
城市 1 分钟销售额	window_start + city	同一个城市在不同分钟是不同结果；同一分钟重放则是同一结果
订单超时告警	order_id + alert_type + action_time	同一笔超时事实可稳定重建；故障重放仍得到同一 alert_id
反例：随机 UUID	每次生成新 UUID	每次重放都变成新主键，数据库根本无法识别是同一业务事实

幂等 Upsert 的作用：允许“重新执行”，但阻止“最终表重复一行”



6. 上机前检查：每次仍然从环境开始

启动 Ubuntu VM → Windows 打开 VS Code → Remote SSH → 打开远程目录 → Terminal。按顺序执行：

```
hostname
pwd
whoami
java -version
mvn -version
echo $KAFKA_HOME
echo $FLINK_HOME
ss -lntp | grep 9092 || true
ss -lntp | grep 3306 || true
$FLINK_HOME/bin/flink --version
```

浏览器检查 Flink Web UI：

```
http://<虚拟机 IP>:8081
```

通过标准：Kafka 9092 正常；MySQL 3306 正常；Flink Web UI 能看到 TaskManager/Slots；Java/Maven 与第 7/8 讲一致。

7. 第一步：初始化 MySQL 结果库和两张表

代码包中已经提供 `sql/09\_init\_mysql.sql`。第一次建议打开文件看清楚字段，再执行：

```
cd ~/bigdata-project/lecture9-external-sinks
mysql -u root -p < sql/09_init_mysql.sql
```

核心表结构：

```
CREATE TABLE city_sales_rt (
  window_start DATETIME NOT NULL,
  window_end   DATETIME NOT NULL,
  city         VARCHAR(64) NOT NULL,
  order_count  BIGINT NOT NULL,
  sales_amount DECIMAL(18,2) NOT NULL,
  PRIMARY KEY (window_start, city)
);

CREATE TABLE order_timeout_alerts (
  alert_id VARCHAR(160) PRIMARY KEY,
  order_id VARCHAR(64) NOT NULL,
  alert_type VARCHAR(32) NOT NULL,
  ...
);
```

为什么主键这么重要：它既是数据库约束，也是“重放时识别同一业务结果”的锚点。没有稳定主键，就无法谈可靠 Upsert。

8. 第二步：创建 4 个干净的 Kafka Topic

```
./09_reset_topics.sh

# 预期包含
orders_sales_l9
city_sales_l9
orders_timeout_l9
order_alerts_l9
```

两个输入 Topic 分别复用第 7 讲和第 8 讲的业务数据；两个输出 Topic 则专门用来验证外部 Sink。

Topic	方向	用途
orders_sales_l9	Source	第 7 讲的城市销售窗口输入
city_sales_l9	Sink	窗口销售结果，Kafka exactly-once
orders_timeout_l9	Source	第 8 讲的订单超时状态机输入
order_alerts_l9	Sink	只保留 TIMEOUT_ALERT，Kafka exactly-once

9. 第三步：代码包目录与 Maven 依赖

```
lecture9-external-sinks/
├── pom.xml
├── src/main/java/edu/project/realtime/
│   ├── CitySalesExternalSinkJob.java
│   └── OrderTimeoutExternalSinkJob.java
├── sql/09_init_mysql.sql
├── data/
│   ├── city_sales_demo.jsonl
│   └── order_timeout_demo.jsonl
├── 09_*.sh / 09_emit_keyed_events.py
└── expected_results.txt
```

pom.xml 比第 8 讲多了两类依赖：JDBC Connector 与 MySQL Driver。Kafka/Flink 版本保持不变。

```
<flink.version>1.20.3</flink.version>
<flink.kafka.connector.version>3.4.0-1.20</flink.kafka.connector.version>
<flink.jdbc.connector.version>3.4.0-1.20</flink.jdbc.connector.version>
<mysql.connector.version>8.4.0</mysql.connector.version>
```

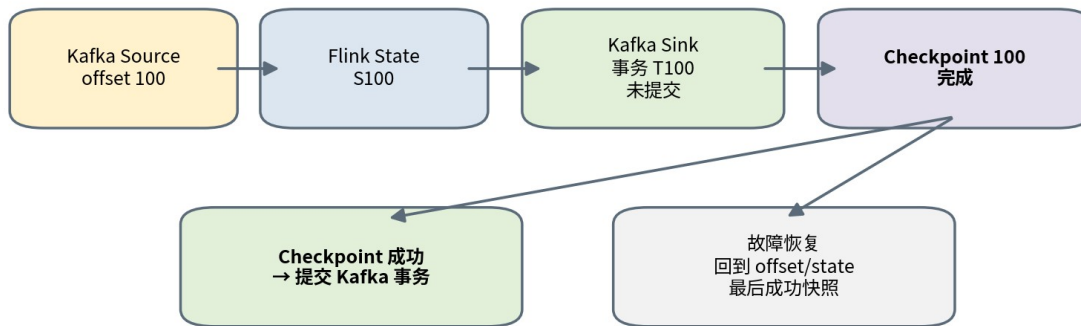
10. 第四步：城市销售结果如何写 Kafka Exactly-once

第 7 讲的计算链不变，变化发生在最后：不再 `print()` 作为主输出，而是构建 KafkaSink。

```
KafkaSink<CitySalesResult> kafkaSink = KafkaSink.<CitySalesResult>builder()
    .setBootstrapServers(BOOTSTRAP)
    .setRecordSerializer(new CitySalesKafkaSerializer())
    .setDeliveryGuarantee(DeliveryGuarantee.EXACTLY_ONCE)
    .setTransactionalIdPrefix("city-sales-l9-")
    .build();

citySales.sinkTo(kafkaSink);
```

Kafka Exactly-once : 事务提交与 Checkpoint 成功绑定



Flink 会让 Kafka 输出事务和 Checkpoint 成功绑定。消费者若使用 `read\_committed`，只看到已经成功提交的事务数据。

可见性延迟：Exactly-once Kafka 结果通常要等 Checkpoint 成功后才对 `read\_committed` 消费者可见。因此“没立刻看到输出”不一定是坏事，先看 Checkpoint 是否完成。

11. Kafka Key 为什么也要设计

CitySalesKafkaSerializer 把 `window\_start|city` 作为 Kafka Key，把 JSON 作为 Value：

```
key = "2026-09-29 10:01:00|Beijing"
value = {"window_start": "...", "city": "Beijing",
        "order_count": 1, "sales_amount": 258.00}
```

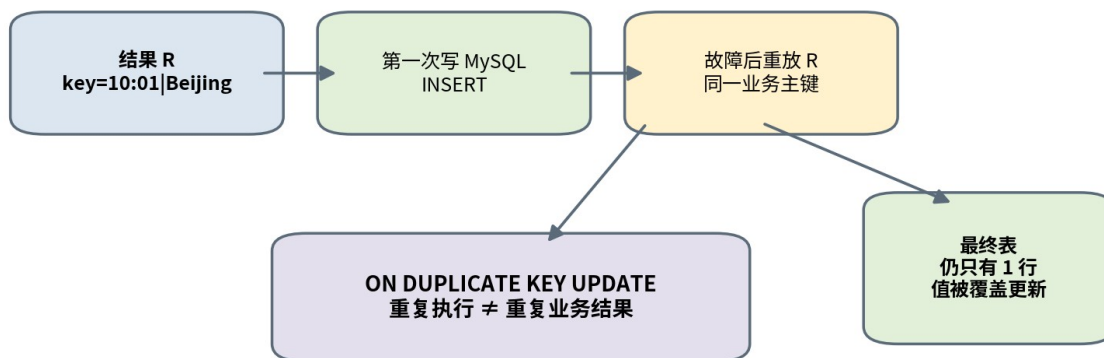
这样下游可以按业务 Key 做 Compact、Upsert 或状态物化，也能让相同业务结果稳定进入同一 Key 语义。

12. 第五步：MySQL 为什么不用普通 INSERT，而用 Upsert

```
INSERT INTO city_sales_rt
(window_start, window_end, city, order_count, sales_amount)
VALUES (?, ?, ?, ?, ?)
ON DUPLICATE KEY UPDATE
window_end=VALUES(window_end),
order_count=VALUES(order_count),
sales_amount=VALUES(sales_amount);
```

普通 JdbcSink 提供 at-least-once：失败恢复时同一输出可能再次执行 SQL。如果是普通 INSERT，可能出现重复行或主键冲突；Upsert 则把同一业务 Key 的重复执行变成覆盖更新。

幂等 Upsert 的作用：允许“重新执行”，但阻止“最终表重复一行”



准确表述：“MySQL 最终表没有重复”不等于 JDBC 链路变成了 XA Exactly-once。我们得到的是业务层幂等最终状态。它和事务 Exactly-once 是不同机制。

13. 第六步：先编译，再设置数据库凭据

```
cd ~/bigdata-project/lecture9-external-sinks
./09_build.sh
```

不要把数据库密码直接写进 Java 源码。当前终端临时设置：

```
export MYSQL_USER=root
export MYSQL_PASSWORD='你的MySQL密码'
export MYSQL_URL='jdbc:mysql://localhost:3306/bigdata_project?useSSL=false&serverTimezone=Asia/Shanghai&allowPublicKeyRetrieval=true'
```

安全习惯：教学 VM 可以用环境变量简化；生产环境应使用 Secret/凭据管理系统，避免密码进入 Git、日志或命令历史。

14. 第七步：运行城市销售 Sink Job

```
./09_run_city_sales.sh
$FLINK_HOME/bin/flink list -r
```

然后发送第 7 讲同一组事件：

```
./09_send_city_sales_demo.sh
```

标准业务结果：

```
[10:01,10:02) Beijing    count=1  sales=258.00
[10:01,10:02) Shanghai  count=1  sales=499.00
[10:02,10:03) Beijing    count=1  sales=118.00
```

15. 第八步：先验证 Kafka Sink

另开一个 VS Code Terminal：

```
./09_consume_city_sales.sh
```

这个脚本显式设置：

```
--consumer-property isolation.level=read_committed
```

如果 KafkaSink 使用 exactly-once 事务，但消费者使用 read_uncommitted，就可能看到尚未提交的事务数据，验证语义就不严谨。

排错：Topic 没输出时先看 Flink Web UI 的 Completed Checkpoints。Exactly-once Kafka 事务通常在 Checkpoint 完成后提交。

16. 第九步：再验证 MySQL 表

```
mysql -u root -p
USE bigdata_project;
SELECT window_start, window_end, city, order_count, sales_amount
FROM city_sales_rt
ORDER BY window_start, city;
```

行数应为 3，内容应与 Kafka 结果完全一致。

六层证据：Source 事件 → Flink Job/Checkpoint → Kafka read_committed → MySQL 主键行 → 标准答案 → 日志/指标。不要只看一个 Sink 就结束。

17. 第十步：运行订单超时告警 Sink Job

先停止城市 Job（避免教学机器资源不足），记下 Job ID：

```
$FLINK_HOME/bin/flink list -r
$FLINK_HOME/bin/flink cancel <CitySalesJobId>
```

再运行告警 Job：

```
./09_run_timeout.sh
./09_send_timeout_demo.sh
```

标准告警只有两条：

```
03002 TIMEOUT_ALERT 2026-09-29 10:16:00
03004 TIMEOUT_ALERT 2026-09-29 10:18:00
```

18. 第十一步：验证告警 Kafka 与 MySQL

```
./09_consume_alerts.sh
```

MySQL 查询：

```
mysql -u root -p bigdata_project < 09_verify_mysql.sql
```

`order\_timeout\_alerts` 应只有 O3002、O3004 两个超时业务事实。O3001 按时支付、O3003 取消、O3999 按时支付，都不应进入告警表。

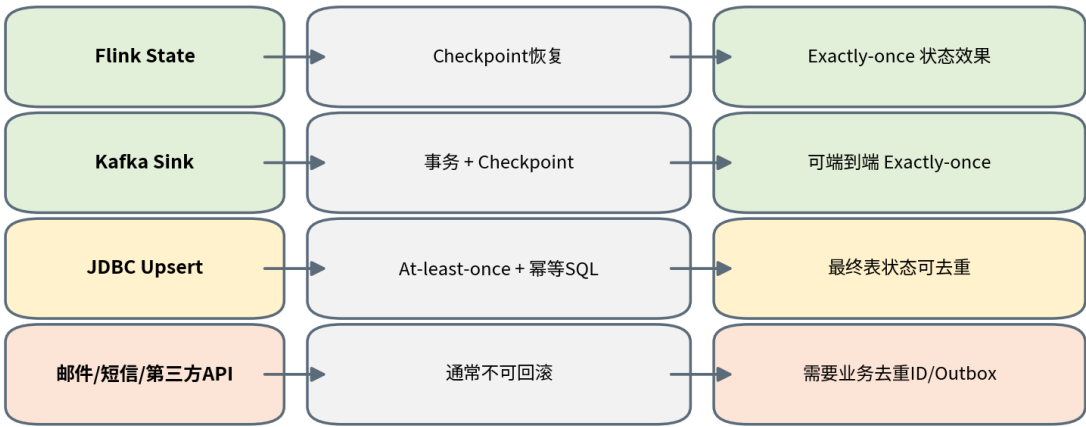
19. 一个非常重要的边界：Sink 幂等不能替你解决“输入本身重复”

假设你把同一批 PAY 事件作为“新的 Kafka 记录”重新发送到 Source Topic。它们有新的 Offset，Flink 会把它们当成新的业务事件；城市窗口聚合可能被真正加两次。MySQL Upsert 只能去重“同一个聚合结果的重复写”，不能推断哪些 Source 事件是业务重复。

结论：Source 去重、业务事件幂等、Flink 状态一致性、Sink 幂等是不同层次。不要指望一个数据库 PRIMARY KEY 解决整个端到端重复问题。

20. 故障重放时，哪些层次分别负责什么

Exactly-once 是一条端到端链路属性，不是给 Flink 打开一个开关就结束



层次	机制	解决的问题
Kafka Source	Offset 参与 Checkpoint	从一致位置重新消费
Flink State/Timer	Checkpoint	恢复计算中的记忆
Kafka Sink	事务 + Checkpoint	已提交输出避免重复可见
MySQL 普通 JDBC	at-least-once + Upsert	重复 SQL 对最终表状态尽量无副作用
业务输入重复	event_id / order_id / 去重状态	必须额外设计，Sink 无法自动替代

21. 为什么本讲暂时不主讲 JDBC XA Exactly-once

Flink JDBC Connector 支持基于 XA 的 exactlyOnceSink，但它要求数据库和 JDBC Driver 正确支持 XA，还涉及连接数、XA 权限、prepared transaction、重试策略等配置。

对当前零基础综合项目，先把“at-least-once + 稳定业务主键 + 幂等 Upsert”真正理解和跑通，更有价值。等你进入生产高一致性专题，再单独做 XA/2PC 实验。

官方细节：当前 JDBC exactlyOnceSink 依赖 XA；官方文档还提醒 exactly-once 模式下要特别注意 retry 与数据库 XA 配置。

22. 本讲完整验收证据链

本讲验收不是“看到一行输出”，而是验证交付语义

1	Kafka Source	输入事件存在
2	Flink Web UI	Job / Checkpoint 正常
3	Kafka Sink	read_committed 可见结果
4	MySQL	主键行数与金额正确
5	重放实验	重复发送后表不增重复行

- 1. Kafka 输入 Topic 能看到测试事件。
- 2. Flink Web UI 中 Job 为 RUNNING，Completed Checkpoints 持续增加。
- 3. city_sales_l9 使用 read_committed 能看到 3 条标准窗口结果。
- 4. MySQL city_sales_rt 恰好 3 行，金额分别为 258/499/118。
- 5. order_alerts_l9 只出现 O3002、O3004 两条超时告警。
- 6. MySQL order_timeout_alerts 只保存同样两条业务事实。
- 7. 你能解释 Kafka Sink 的 exactly-once 与 MySQL 幂等 Upsert 为什么不是同一种机制。

23. 最常见的 10 类错误与固定排查顺序

现象	优先检查	典型原因
Kafka 输出看不到	Checkpoint / isolation.level	事务还未提交或消费者不是 read_committed
MySQL Access denied	MYSQL_USER/PASSWORD	凭据错误
No suitable driver	pom / shaded jar	MySQL driver 未打入 JAR
Table does not exist	09_init_mysql.sql	数据库/表未初始化
Duplicate key 报错	SQL 是否 Upsert	使用了普通 INSERT
MySQL 行数越来越多	业务主键	主键使用随机值或字段不稳定
Kafka 有重复可见	delivery guarantee/事务前缀	未启用 exactly-once 或消费隔离不对
窗口金额翻倍	Source 事件重复	这是输入层去重问题，不是 Sink Upsert
Checkpoint 一直失败	Sink/事务/网络	查 checkpoint failure reason
Job RUNNING 但表不更新	Source/Watermark/Sink	RUNNING 不代表业务链路完整

固定排查顺序：环境 → Source → 计算结果 → Checkpoint → Kafka Sink → MySQL Sink → 业务主键 → 标准答案。

24. 练习题

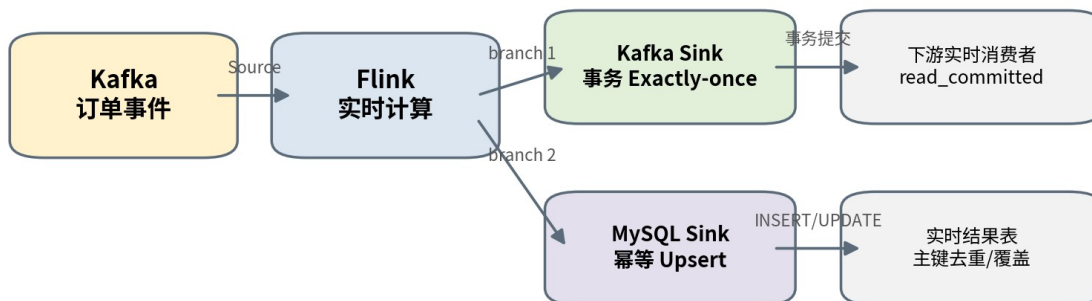
1. 为什么 Flink State exactly-once 不能自动推出 MySQL Sink exactly-once?
2. Kafka exactly-once Sink 为什么要求消费者关注 read_committed?
3. 为什么 city_sales_rt 的主键是 (window_start, city)，不能只用 city?
4. 使用随机 UUID 作为 alert_id，为什么会破坏幂等重放?
5. MySQL Upsert 能解决“同一个 Source PAY 事件被重新生产一遍”的重复统计吗？为什么？
6. 如果业务真的要求 MySQL 数据库事务级 exactly-once，本讲方案下一步应研究什么？

25. 参考答案

1. Checkpoint 首先保证 Flink 管理状态的一致恢复；外部 Sink 必须自己参与 checkpoint/事务协议，否则重试仍可能重复写。
2. Kafka 事务中未提交记录不应被当作最终业务结果；read_committed 只暴露已经成功提交的事务数据。
3. 同一个城市每分钟都有新结果；只用 city 会让新窗口覆盖旧窗口。window_start + city 才稳定标识一个窗口结果。
4. 同一业务事实每次重放都会生成不同主键，数据库会把它们当成不同记录，无法去重。
5. 不能。重新生产的 Source 事件具有新的 Kafka Offset，并会重新参与窗口聚合。需要在输入/业务层额外做 event_id 去重。
6. 研究 JdbcSink.exactlyOnceSink、XA/2PC、数据库 XA 能力、连接与重试配置，并做故障注入验证。

26. 本讲最终心智模型

第9讲端到端：同一条实时结果，进入不同外部系统时，语义保证并不相同



到这里你应该把 Sink 理解成“可靠性协议的最后一公里”，而不是把结果 print 改成 insert 就结束。

一句话总结：Kafka 可以通过事务和 Checkpoint 做真正的交付 exactly-once；MySQL 普通 JDBC 更常见的是 at-least-once + 稳定业务主键 + 幂等 Upsert。端到端可靠性必须逐段分析。